C Neural Network Benchmark - MNIST training set

A Serial and pthread-based parallel training experiment in C

Objective

This benchmarking report briefly outlines how configurable multithreaded training affects the runtime of my C neural-network library built from scratch. It compares training performance across two MNIST dataset sizes over 5 different thread worker counts, with particular interest in the speedup and parallel efficiency achieved.

Setup

Hardware	Intel Core i5-9500 @ 3.00 GHz; 6 physical cores / 6 logical CPUs; 32 GiB RAM
Operating System	Ubuntu Linux v 26.04
Compiler	GCC 15.2.0, x86_64-linux-gnu
Model	Neural network with 784 input features, 3 layers, 1 input, 1 hidden layer of 32 ReLU neurons, 1 output layer with 10 softmax output neurons; categorical cross-entropy loss.
Training	MNIST; 5,000 and 60,000 samples; 10 epochs per run, learning rate 0.2; thread counts 1, 2, 4, 6, and 12
Repetitions	10 runs per configuration, 100 runs in total. Jobs were randomised before execution using a fixed random seed.
Timing	Total runtime from reading in the data, to creating the model, training and evaluation. As well as training time specific measurements due to training being the parallel section under investigation.

The machine used has six logical CPUs, so for configurations above six workers were purposely used to observe any changes to oversubscribed units.

Results

Tables 1 & 2 report the median training times across the 10 repetitions for each of the experiment configurations. Speedup is the single-threaded median divided by the multithreaded median for each of the respective dataset sizes. Parallel efficiency is the speedup divided by the number of worker threads for each thread group.

Table 1 - 5,000 MNIST samples over 10 runs per configuration

Threads	Median train (s)	Speedup	Efficiency	Median total (s)
1	9.210	1.00×	100.0%	10.577
2	4.650	1.98×	99.0%	6.017
4	2.482	3.71×	92.8%	3.849
6	1.689	5.45×	90.9%	3.056
12	1.772	5.20×	43.3%	3.145

Table 2 - 60,000 MNIST samples over 10 runs per configuration

Threads	Median train (s)	Speedup	Efficiency	Median total (s)
1	110.571	1.00×	100.0%	116.109
2	55.822	1.98×	99.0%	61.358
4	28.441	3.89×	97.2%	33.980
6	20.151	5.49×	91.5%	25.694
12	20.273	5.45×	45.5%	25.815

From both table 1 and 2 it is evident that parallel training substantially reduced runtime for both dataset sizes. With 6 threads, the median training time fell from 9.21s to 1.68s for 5,000 samples (5.45x speedup, 90.9% efficiency), and from 110.57s to 20.15s for 60,000 samples (5.49x speedup, 91.5% efficiency). The similarities across both dataset sizes suggests that the parallel implementation maintains comparable performance as the workload increases. Scaling the load didn't have a negative effect in training time and shows for this architecture (3 layers, 1 hidden ReLu with 32 neurons, 10 output classes) a linear runtime growth.

Figure 1 shows the median epoch time for each different thread count and is consistent with median train and total times showing a similar speedup with a single epoch on 60,000 samples taking more than 10s on a single thread and a touch above 2s for 6 threads, with each added thread level minimising the training time. All three figures show that performance plateaus beyond 6 threads. On 60,000 samples, median runtime was 20.15 seconds at 6 threads and 20.27s at 12 threads, this was the same for 5,000 samples as well and is consistent with expectations that parallel coordination costs start to creep in the more threads or processes you add.

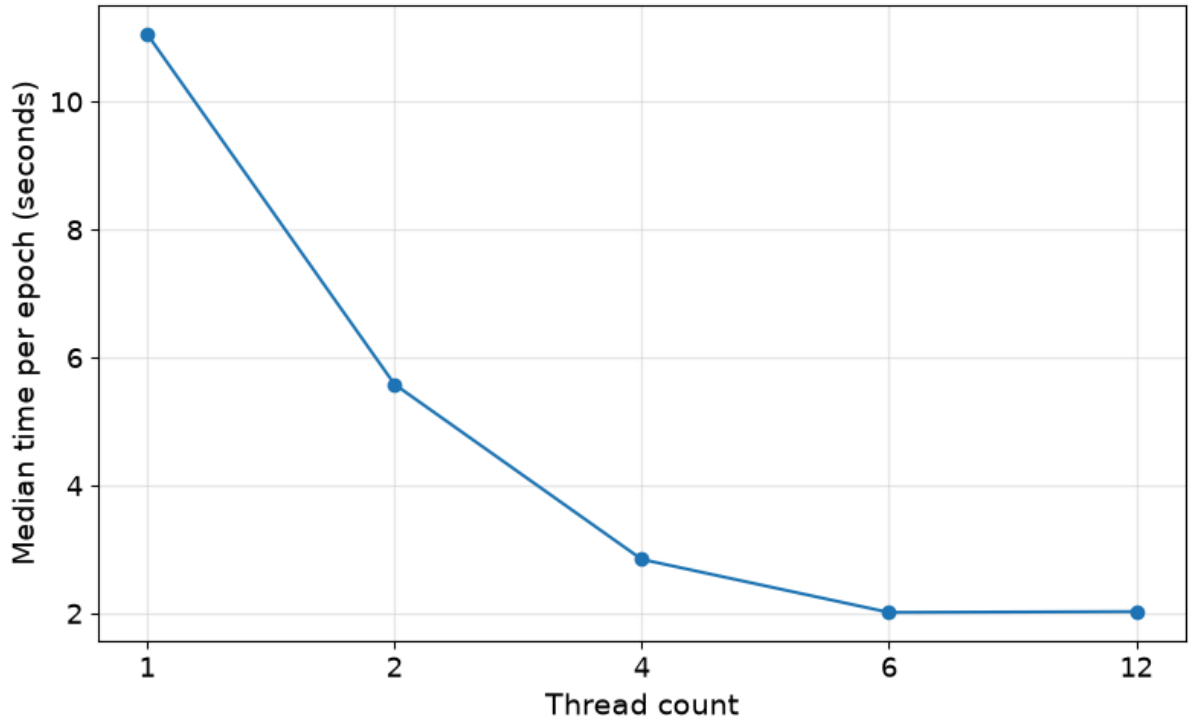


Figure 1 - Median per epoch runtime on 60,000 MNIST samples.

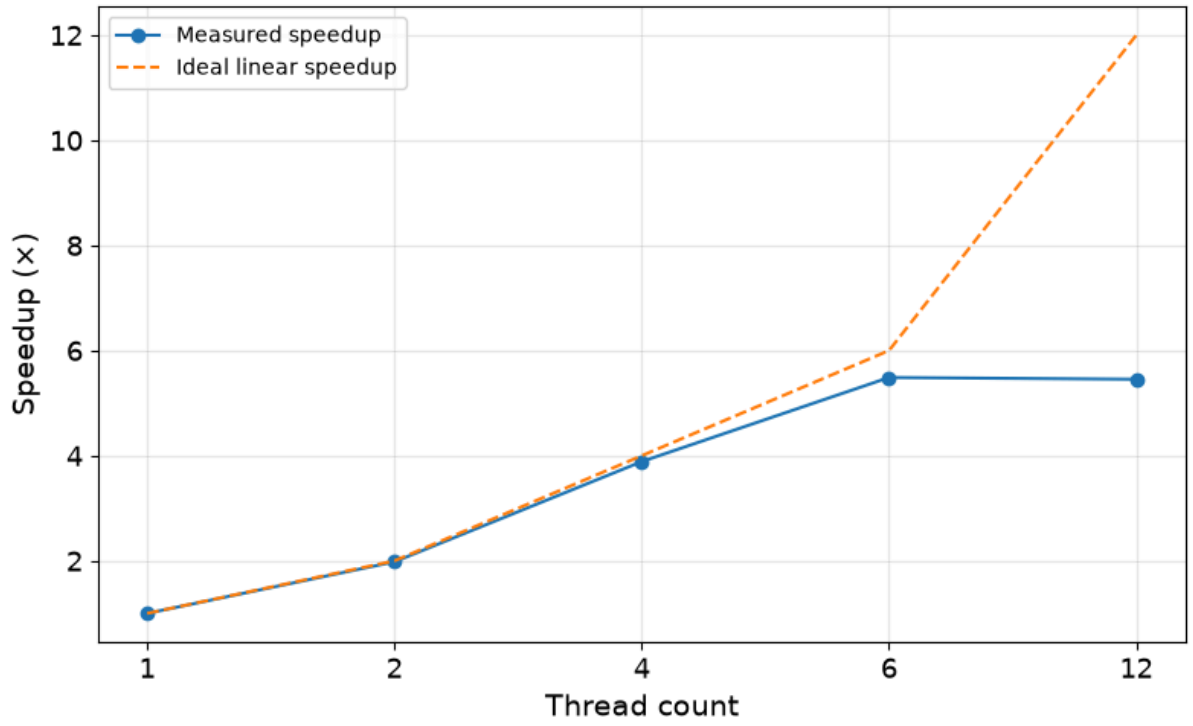


Figure 2 - Measured speed up on 60,000 MNIST samples compared with ideal linear scaling

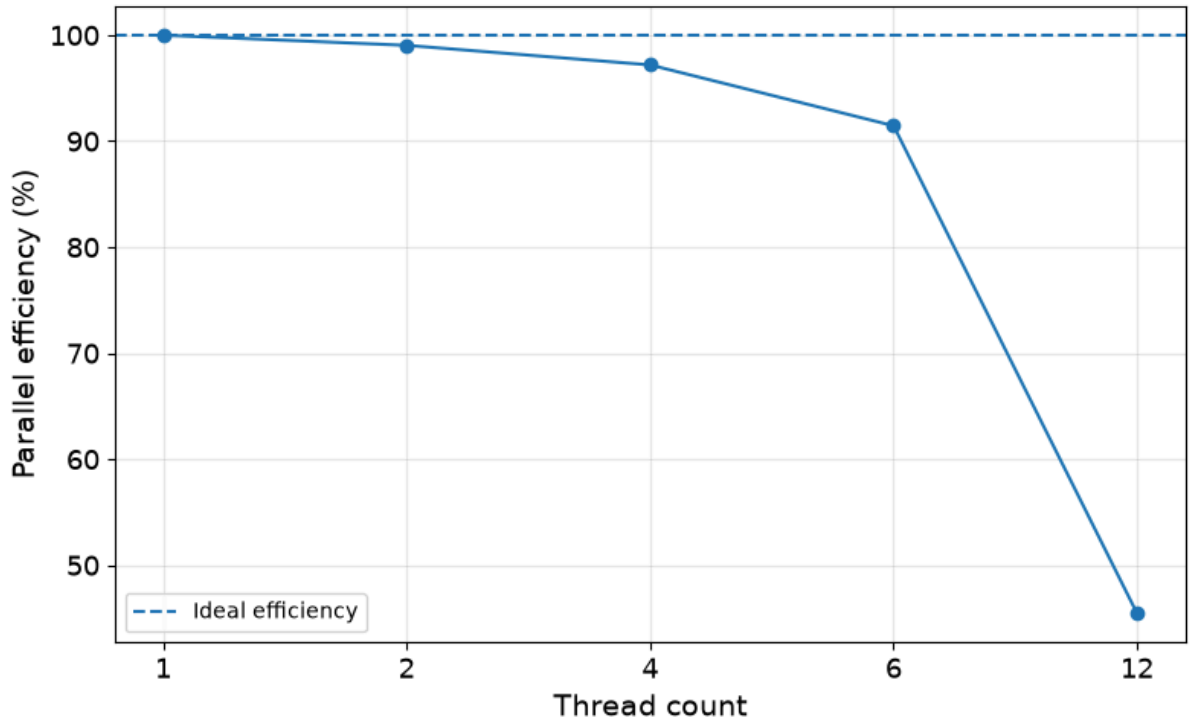


Figure 3 - Parallel efficiency on 60,000 MNIST samples

Conclusion

Overall these results show that parallelising training for my C Neural network library significantly helped speed up training and reduce total runtime for trivial experiments. Specifically, on the tested six-core Intel i5-9500 system, 6 worker threads produced the lowest median training time among the test configurations, delivering approximately 5.5x speedup with around 91% parallel CPU efficiency. Increasing the worker count beyond the available CPUs shows that no extra advantages are made, though this particular set of experiments did not exhaust all possibilities in regards to this fact.

Although these gains were made for the library, the results here are limited to this machine and the implementation and architecture tested. This benchmarkings purpose was only to measure runtimes and the effect of parallelising the training portion of neural net library and not model predictive quality. Though the results shown are promising, they should be taken as a loose guideline with individual tests undertaken for different dataset, architectures and machines.

Reproducibility note

The benchmark comprised 100 runs: 2 sample sizes × 6 thread counts × 10 repetitions. The benchmark script and source CSV can be found in the examples/mnist/benchmarking folder under the names benchmark.py and mnist_benchmarking.csv respectively. Median timings were used to reduce sensitivity to individual unusually fast or slow runs.